AI Analytics Platform - Complete Project Documentation

Table of Contents

---

1. Executive Summary

1.1 Project Vision

Build a **generic, AI-powered analytics platform** that enables businesses across industries to gain intelligent insights from their data with minimal technical overhead. The platform is designed for easy penetration into wide industry sectors without major customization.

1.2 Key Objectives

- **Zero-Day Productivity**: Any company can start using the application on day 1 with ready data sources
- **Industry Agnostic**: Works across retail, healthcare, finance, manufacturing, etc.
- **Progressive Intelligence**: System learns and improves with each client engagement
- **Scalable Architecture**: Start small (SQLite), grow to enterprise (PostgreSQL)
- **Data Security**: Enterprise-grade security even in lightweight deployments

1.3 Target Users

User Type	Use Case	Scale
Small Businesses	Basic analytics, expense tracking	1-10 users, <10GB data
Mid-Market	Department analytics, operational insights	10-100 users, 10-100GB data
Enterprise	Multi-tenant, complex analytics	100+ users, 100GB+ data

1.4 Competitive Advantages

1. **Template-Based Intelligence**: Pre-built industry knowledge accelerates deployment
2. **Dual Deployment Model**: Small (SQLite/DuckDB) → Enterprise (PostgreSQL)
3. **AI-Powered Mapping**: Automatic column detection and semantic understanding
4. **Three-Tier Output**: Reports, Natural Language Insights, Interactive Chatbot

5. **No Vendor Lock-in**: Works with simple CSV/JSON exports

---

2. Project Overview

2.1 Problem Statement

Current Challenges:

- Traditional BI tools require extensive setup and configuration
- Custom analytics solutions are expensive and time-consuming
- Data integration is complex and error-prone
- Insights require specialized data science expertise
- Scaling from small to enterprise is painful

Our Solution: An intelligent analytics platform that:

- Understands data context through industry templates
- Automatically maps and validates data
- Generates insights using AI/ML
- Scales seamlessly from single-user to enterprise
- Maintains security throughout the journey

2.2 Core Value Proposition

```
Traditional Approach              Our Approach
_____________________             _________________

1. Hire data engineer             1. Upload CSV/JSON
2. Build data warehouse           2. Select industry template
3. Create ETL pipelines           3. Review auto-mapping
4. Develop dashboards             4. Get instant insights
5. Maintain infrastructure        5. Scale as needed

Time: 3-6 months                  Time: Day 1
Cost: $50K-$200K                  Cost: Subscription
```

2.3 Key Features Summary

Data Integration

- CSV, JSON, TXT file uploads
- Automatic schema detection
- Intelligent column mapping
- Data validation & sanitization
- Multi-source relationship detection

Intelligence Layer

- Industry-specific templates
- Semantic column understanding
- ML-based anomaly detection
- Predictive analytics
- Natural language generation

Output Types

- **Type 1**: Predefined industry reports with KPIs
- **Type 2**: Natural language insights (executive summaries)
- **Type 3**: Interactive AI chatbot for ad-hoc queries

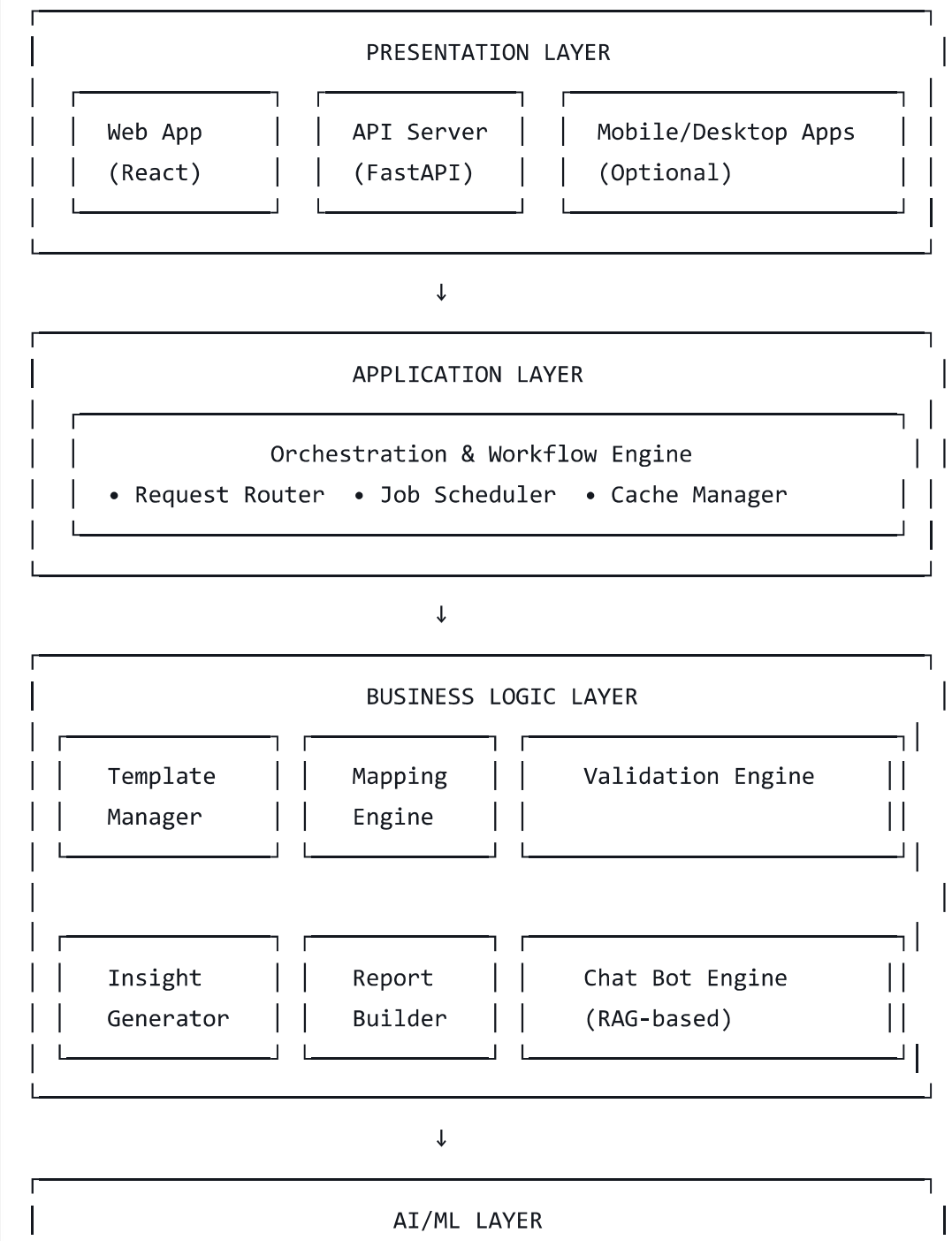
Scalability

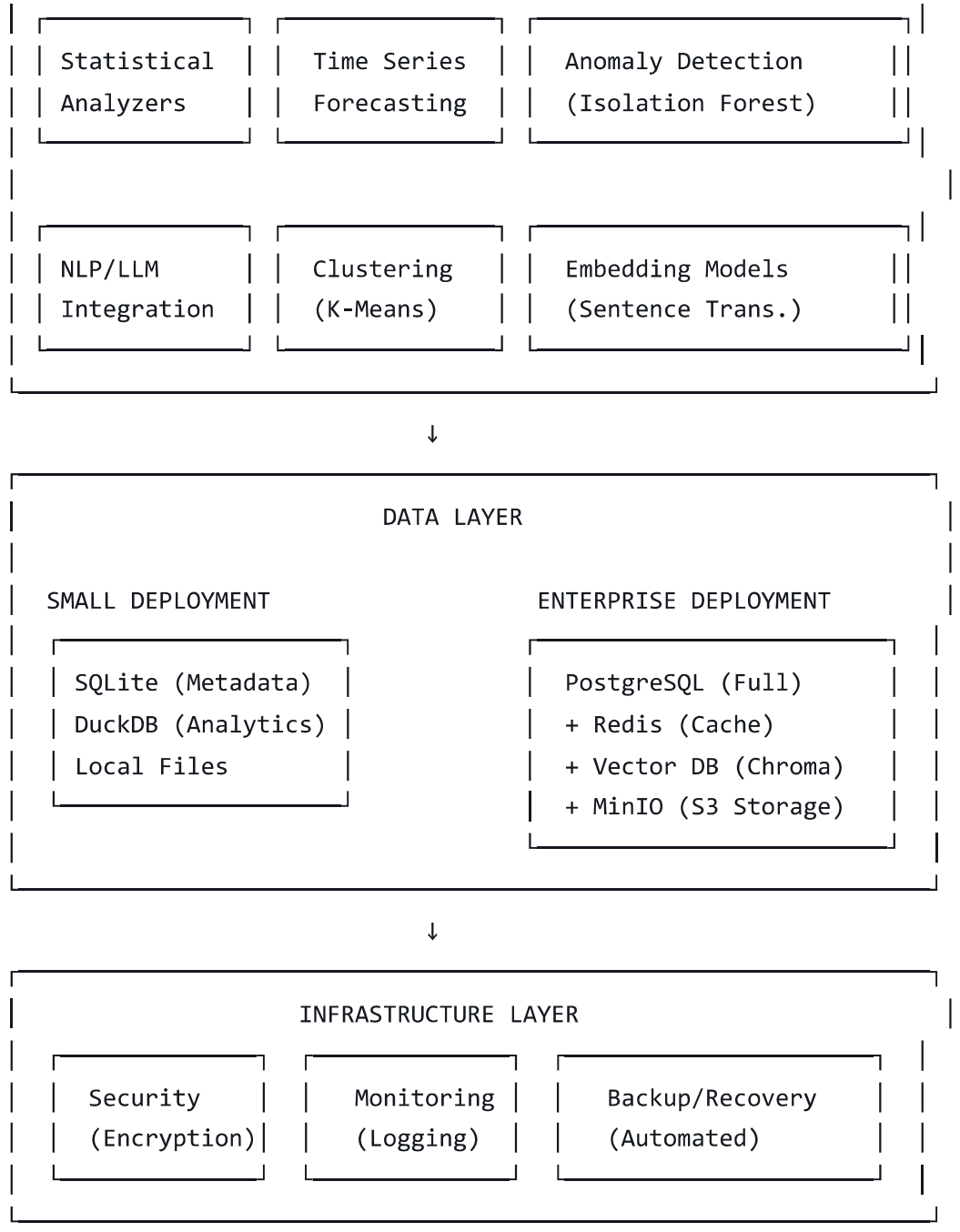
- Small: SQLite + DuckDB (no server needed)
- Enterprise: PostgreSQL + Redis + Vector DB
- One-command migration between tiers

---

3. System Architecture

3.1 High-Level Architecture





3.2 Component Architecture

3.2.1 Data Ingestion Flow

```
User Upload (CSV/JSON)
         ↓
File Validation (size, format, encoding)
         ↓
Schema Detection (columns, types, patterns)
         ↓
Template Selection (industry-based)
         ↓
```

```
Column Mapping (4-layer intelligence)
    ├── Layer 1: Exact name matching
    ├── Layer 2: Semantic similarity (embeddings)
    ├── Layer 3: Content-based inference
    └── Layer 4: Historical learning
         ↓
User Confirmation (ambiguous mappings)
         ↓
Data Validation (types, ranges, business rules)
         ↓
Data Enrichment (derived columns)
         ↓
Storage (Database + File System)
         ↓
Insight Generation (triggered)
```

3.2.2 Insight Generation Flow

```
Data Available
         ↓
Template Insight Rules Loaded
         ↓
Parallel Analysis Execution
    ├── Statistical Analysis
    ├── Temporal Pattern Detection
    ├── Anomaly Detection
    ├── Comparative Analysis
    ├── Predictive Modeling
    └── Text Analysis (if applicable)
         ↓
Insight Ranking (by business value)
         ↓
Natural Language Generation
         ↓
Storage (Cache + Database)
         ↓
Presentation (Reports/Chat/API)
```

3.3 Technology Stack

3.3.1 Backend

python

```
# Core Framework
FastAPI==0.109.0           # Modern, async API framework
Uvicorn==0.27.0            # ASGI server
Pydantic==2.5.0            # Data validation

# Database
SQLAlchemy==2.0.25         # ORM (database agnostic)
alembic==1.13.0            # Database migrations
duckdb==0.9.2              # Analytics database
sqlcipher3==0.5.0          # Encrypted SQLite
psycopg2-binary==2.9.9     # PostgreSQL driver (enterprise)

# Data Processing
pandas==2.1.4              # Data manipulation
polars==0.20.0             # Fast dataframes
numpy==1.26.3              # Numerical computing
great-expectations==0.18.8  # Data validation

# ML/AI
scikit-learn==1.4.0        # Classical ML
prophet==1.1.5             # Time series forecasting
sentence-transformers==2.3.1  # Embeddings
langchain==0.1.0           # LLM orchestration
anthropic==0.18.0          # Claude API
openai==1.10.0             # OpenAI API (optional)

# Task Queue
celery==5.3.4              # Background jobs
redis==5.0.1               # Message broker + cache

# Utilities
python-multipart==0.0.6    # File uploads
python-jose==3.3.0         # JWT tokens
passlib==1.7.4             # Password hashing
cryptography==42.0.0       # Encryption
```

3.3.2 Frontend

json

```
{
  "dependencies": {
    "react": "^18.2.0",
```

```
    "typescript": "^5.3.0",
    "next.js": "^14.0.0",
    "tailwindcss": "^3.4.0",

    "recharts": "^2.10.0",
    "plotly.js": "^2.27.0",
    "ag-grid-react": "^31.0.0",

    "@tanstack/react-query": "^5.17.0",
    "axios": "^1.6.0",
    "zustand": "^4.4.0",

    "react-hook-form": "^7.49.0",
    "zod": "^3.22.0"
  }
}
```

3.3.3 DevOps

yaml

```
# Docker
- Docker Engine 24.0+
- Docker Compose 2.23+

# Monitoring
- Prometheus (metrics)
- Grafana (visualization)
- Sentry (error tracking)

# CI/CD
- GitHub Actions
- Docker Registry

# Testing
- pytest
- pytest-cov
- pytest-asyncio
```

---

4. Technical Specifications

4.1 Data Processing Pipeline

4.1.1 Template System

Template Structure (JSON Schema)

json

```
{
  "template_id": "uuid",
  "industry": "retail",
  "sub_domain": "e-commerce",
  "version": "2.1.0",
  "confidence_score": 0.92,

  "schemas": {
    "sales_transaction": {
      "description": "Point of sale transaction data",
      "columns": {
        "transaction_id": {
          "aliases": ["order_id", "sale_id", "txn_id", "receipt_no"],
          "data_type": "string",
          "semantic_type": "identifier",
          "required": true,
          "validation": {
            "regex": "^[A-Z0-9]{8,12}$",
            "unique": true
          },
          "description": "Unique transaction identifier"
        },
        "transaction_date": {
          "aliases": ["date", "sale_date", "order_date", "timestamp"],
          "data_type": "date",
          "semantic_type": "temporal",
          "required": true,
          "validation": {
            "min_date": "2020-01-01",
            "max_date": "today"
          }
        },
        "amount": {
          "aliases": ["total", "sale_amount", "revenue", "price"],
          "data_type": "numeric",
          "semantic_type": "currency",
          "required": true,
          "validation": {
            "min": 0,
            "max": 1000000
```

```
      }
    },
    "category": {
      "aliases": ["product_category", "department", "type"],
      "data_type": "string",
      "semantic_type": "categorical",
      "required": false,
      "validation": {
        "enum": ["electronics", "clothing", "food", "other"]
      }
    }
  },

  "relationships": {
    "customer_data": {
      "join_on": "customer_id",
      "type": "many_to_one",
      "required": false
    },
    "product_catalog": {
      "join_on": "product_id",
      "type": "many_to_one",
      "required": false
    }
  },

  "enrichments": [
    {
      "name": "day_of_week",
      "source": "transaction_date",
      "function": "extract_dow"
    },
    {
      "name": "is_weekend",
      "source": "transaction_date",
      "function": "is_weekend"
    },
    {
      "name": "quarter",
      "source": "transaction_date",
      "function": "extract_quarter"
    }
  ]
```

```
    }
  },

  "insight_rules": {
    "revenue_trend": {
      "type": "time_series_analysis",
      "priority": "high",
      "required_columns": ["transaction_date", "amount"],
      "parameters": {
        "aggregation": "sum",
        "frequency": "monthly",
        "ml_model": "prophet_forecast"
      },
      "output": {
        "title": "Revenue Trend Analysis",
        "description": "Monthly revenue trends with forecast",
        "visualization": "line_chart"
      }
    },
    "top_categories": {
      "type": "ranking",
      "priority": "medium",
      "required_columns": ["category", "amount"],
      "parameters": {
        "aggregation": "sum",
        "top_n": 10
      }
    },
    "anomaly_detection": {
      "type": "statistical_anomaly",
      "priority": "high",
      "required_columns": ["transaction_date", "amount"],
      "parameters": {
        "method": "isolation_forest",
        "threshold": 0.05
      }
    }
  },

  "reports": {
    "executive_summary": {
      "sections": [
        {
```

```
          "type": "kpi_cards",
          "metrics": ["total_revenue", "growth_rate", "avg_transaction"]
        },
        {
          "type": "trend_chart",
          "metric": "revenue",
          "period": "monthly"
        },
        {
          "type": "top_insights",
          "count": 5,
          "priority_filter": "high"
        }
      ],
      "export_formats": ["pdf", "pptx", "html"]
    }
  }
}
```

4.1.2 Intelligent Mapping Algorithm
Four-Layer Mapping System

python

```python
class IntelligentMapper:
    """
    Layer 1: Exact Matching (100% confidence)
    Layer 2: Semantic Similarity (70-95% confidence)
    Layer 3: Content Analysis (60-85% confidence)
    Layer 4: Historical Learning (50-90% confidence)
    """

    def map_columns(self, df, template):
        mappings = {}

        # Layer 1: Exact match
        for col in df.columns:
            if col in template.column_names:
                mappings[col] = {
                    'template_column': col,
                    'confidence': 1.0,
                    'method': 'exact'
                }
```

```
        # Layer 2: Semantic similarity
        unmapped = [c for c in df.columns if c not in mappings]
        semantic_matches = self._semantic_match(unmapped, template)
        mappings.update(semantic_matches)

        # Layer 3: Content-based
        still_unmapped = [c for c in df.columns if c not in mappings]
        content_matches = self._analyze_content(df[still_unmapped], template)
        mappings.update(content_matches)

        # Layer 4: Historical patterns
        historical_matches = self._check_history(df.columns, template)
        for col, match in historical_matches.items():
            if col not in mappings or mappings[col]['confidence'] < match['confidence']:
                mappings[col] = match

        return mappings
```

Content Analysis Patterns

python

```
DETECTION_PATTERNS = {
    'email': r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$',
    'phone': r'^[\d\s\-\+\(\)]{10,}$',
    'url': r'^https?://[^\s]+$',
    'date': [
        r'^\d{4}-\d{2}-\d{2}$',  # YYYY-MM-DD
        r'^\d{2}/\d{2}/\d{4}$',  # MM/DD/YYYY
        r'^\d{2}-\d{2}-\d{4}$'   # DD-MM-YYYY
    ],
    'currency': r'^\$?\d+(\.\d{2})?$',
    'percentage': r'^\d+(\.\d+)?%?$',
    'zip_code': r'^\d{5}(-\d{4})?$',
    'ssn': r'^\d{3}-\d{2}-\d{4}$'
}
```

4.2 Insight Generation Algorithms

4.2.1 Statistical Insights

python

```python
class StatisticalAnalyzer:
    def analyze(self, df, column, category_col=None):
        insights = []

        # Basic statistics
        insights.append({
            'type': 'summary_statistics',
            'metric': column,
            'values': {
                'mean': df[column].mean(),
                'median': df[column].median(),
                'std': df[column].std(),
                'min': df[column].min(),
                'max': df[column].max()
            }
        })

        # Distribution analysis
        if self._is_normal_distribution(df[column]):
            insights.append({
                'type': 'distribution',
                'finding': 'normal_distribution',
                'confidence': 0.95
            })

        # Category comparison
        if category_col:
            category_stats = df.groupby(category_col)[column].agg(['mean', 'sum', 'count'])
            top_category = category_stats['sum'].idxmax()

            insights.append({
                'type': 'categorical_ranking',
                'finding': f'{top_category} accounts for highest {column}',
                'value': category_stats.loc[top_category, 'sum'],
                'percentage': category_stats.loc[top_category, 'sum'] / df[column].sum()
            })

        return insights
```

4.2.2 Time Series Analysis

python

```python
from prophet import Prophet

class TimeSeriesAnalyzer:
    def forecast(self, df, date_col, value_col, periods=90):
        # Prepare data for Prophet
        prophet_df = df[[date_col, value_col]].rename(
            columns={date_col: 'ds', value_col: 'y'}
        )

        # Fit model
        model = Prophet(
            yearly_seasonality=True,
            weekly_seasonality=True,
            daily_seasonality=False
        )
        model.fit(prophet_df)

        # Forecast
        future = model.make_future_dataframe(periods=periods)
        forecast = model.predict(future)

        # Extract insights
        insights = {
            'type': 'forecast',
            'predictions': forecast[['ds', 'yhat', 'yhat_lower', 'yhat_upper']].tail(periods
            'trend': self._detect_trend(forecast),
            'seasonality': model.seasonalities
        }

        return insights

    def _detect_trend(self, forecast):
        recent_trend = forecast['trend'].tail(30).mean()
        earlier_trend = forecast['trend'].head(-30).mean()

        if recent_trend > earlier_trend * 1.1:
            return 'increasing'
        elif recent_trend < earlier_trend * 0.9:
            return 'decreasing'
        else:
            return 'stable'
```

4.2.3 Anomaly Detection

python

```python
from sklearn.ensemble import IsolationForest

class AnomalyDetector:
    def detect(self, df, columns, contamination=0.05):
        # Prepare features
        X = df[columns].fillna(df[columns].median())

        # Fit Isolation Forest
        clf = IsolationForest(
            contamination=contamination,
            random_state=42
        )
        predictions = clf.fit_predict(X)

        # Identify anomalies
        anomalies = df[predictions == -1].copy()
        anomalies['anomaly_score'] = clf.score_samples(X[predictions == -1])

        insights = {
            'type': 'anomaly_detection',
            'count': len(anomalies),
            'percentage': len(anomalies) / len(df) * 100,
            'samples': anomalies.nlargest(10, 'anomaly_score').to_dict('records')
        }

        return insights
```

4.3 Natural Language Generation

python

```python
class InsightNarrator:
    def __init__(self, llm_client):
        self.llm = llm_client

    def generate_narrative(self, insights, template, data_context):
        """Convert structured insights to natural language"""

        prompt = f"""
You are a business analyst presenting insights to executives.
```

```
Context:
- Industry: {template.industry}
- Data Period: {data_context['date_range']}
- Data Size: {data_context['row_count']} records

Insights Found:
{json.dumps(insights, indent=2)}

Task: Generate a concise executive summary (3-5 paragraphs) that:
1. Highlights the top 3 most important findings
2. Explains trends and patterns in business terms
3. Provides actionable recommendations
4. Uses clear, non-technical language

Format:
- Start with a compelling headline
- Use short paragraphs
- Include specific numbers and percentages
- End with clear next steps
"""

        narrative = self.llm.complete(prompt, max_tokens=800)

        return {
            'title': self._extract_headline(narrative),
            'summary': narrative,
            'key_takeaways': self._extract_takeaways(narrative)
        }
```

---

5. Database Design

5.1 Small Deployment (SQLite + DuckDB)

5.1.1 SQLite Schema (Metadata)

sql

```
-- Application Configuration
CREATE TABLE app_config (
    config_key VARCHAR(100) PRIMARY KEY,
    config_value TEXT, -- JSON stored as text
    environment VARCHAR(20),
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```sql
-- Tenants (Organizations)
CREATE TABLE tenants (
    tenant_id TEXT PRIMARY KEY,
    company_name VARCHAR(200) NOT NULL,
    industry VARCHAR(100),
    subscription_tier VARCHAR(50),
    encryption_key_hash VARCHAR(255),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    is_active BOOLEAN DEFAULT 1
);

-- Users
CREATE TABLE users (
    user_id TEXT PRIMARY KEY,
    tenant_id TEXT NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    password_hash VARCHAR(255),
    role VARCHAR(50),
    preferences TEXT, -- JSON
    last_login TIMESTAMP,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (tenant_id) REFERENCES tenants(tenant_id)
);

-- Industry Templates
CREATE TABLE industry_templates (
    template_id TEXT PRIMARY KEY,
    industry_name VARCHAR(100) NOT NULL,
    sub_domain VARCHAR(100),
    version VARCHAR(20),
    confidence_score INTEGER, -- 0-100
    is_active BOOLEAN DEFAULT 1,
    template_schema TEXT, -- JSON
    insight_rules TEXT, -- JSON
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Template Columns
CREATE TABLE template_columns (
    column_id TEXT PRIMARY KEY,
    template_id TEXT NOT NULL,
```

```
    column_name VARCHAR(100),
    semantic_type VARCHAR(50),
    data_type VARCHAR(50),
    aliases TEXT, -- JSON array
    validation_rules TEXT, -- JSON
    is_required BOOLEAN DEFAULT 0,
    description TEXT,
    FOREIGN KEY (template_id) REFERENCES industry_templates(template_id)
);

-- Data Sources
CREATE TABLE data_sources (
    source_id TEXT PRIMARY KEY,
    tenant_id TEXT NOT NULL,
    source_name VARCHAR(200),
    source_type VARCHAR(50),
    file_path VARCHAR(500),
    file_size_bytes INTEGER,
    row_count INTEGER,
    column_count INTEGER,
    detected_schema TEXT, -- JSON
    mapped_template_id TEXT,
    mapping_confidence INTEGER, -- 0-100
    status VARCHAR(50),
    upload_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (tenant_id) REFERENCES tenants(tenant_id),
    FOREIGN KEY (mapped_template_id) REFERENCES industry_templates(template_id)
);

-- Column Mappings
CREATE TABLE column_mappings (
    mapping_id TEXT PRIMARY KEY,
    source_id TEXT NOT NULL,
    template_id TEXT NOT NULL,
    source_column VARCHAR(100),
    template_column VARCHAR(100),
    mapping_method VARCHAR(50),
    confidence_score INTEGER,
    transformation_rule TEXT, -- JSON
    is_confirmed BOOLEAN DEFAULT 0,
    confirmed_by TEXT,
    confirmed_at TIMESTAMP,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

```
    FOREIGN KEY (source_id) REFERENCES data_sources(source_id),
    FOREIGN KEY (template_id) REFERENCES industry_templates(template_id)
);

-- Insights
CREATE TABLE insights (
    insight_id TEXT PRIMARY KEY,
    tenant_id TEXT NOT NULL,
    source_id TEXT NOT NULL,
    insight_type VARCHAR(50),
    category VARCHAR(100),
    title VARCHAR(500),
    description TEXT,
    narrative TEXT,
    metrics TEXT, -- JSON
    visualization_data TEXT, -- JSON
    confidence_score INTEGER,
    importance_score INTEGER,
    data_period_start DATE,
    data_period_end DATE,
    is_current BOOLEAN DEFAULT 1,
    generated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (tenant_id) REFERENCES tenants(tenant_id),
    FOREIGN KEY (source_id) REFERENCES data_sources(source_id)
);

-- Chat Sessions
CREATE TABLE chat_sessions (
    session_id TEXT PRIMARY KEY,
    tenant_id TEXT NOT NULL,
    user_id TEXT NOT NULL,
    data_context TEXT, -- JSON
    started_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    last_activity TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    is_active BOOLEAN DEFAULT 1,
    FOREIGN KEY (tenant_id) REFERENCES tenants(tenant_id),
    FOREIGN KEY (user_id) REFERENCES users(user_id)
);

-- Chat Messages
CREATE TABLE chat_messages (
    message_id TEXT PRIMARY KEY,
    session_id TEXT NOT NULL,
```

```
    role VARCHAR(20),
    content TEXT,
    query_type VARCHAR(50),
    sql_executed TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (session_id) REFERENCES chat_sessions(session_id)
);

-- Audit Logs
CREATE TABLE audit_logs (
    log_id INTEGER PRIMARY KEY AUTOINCREMENT,
    tenant_id TEXT NOT NULL,
    user_id TEXT,
    action_type VARCHAR(50),
    resource_type VARCHAR(50),
    resource_id TEXT,
    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (tenant_id) REFERENCES tenants(tenant_id),
    FOREIGN KEY (user_id) REFERENCES users(user_id)
);

-- Indexes
CREATE INDEX idx_tenants_industry ON tenants(industry);
CREATE INDEX idx_users_tenant ON users(tenant_id);
CREATE INDEX idx_templates_industry ON industry_templates(industry_name);
CREATE INDEX idx_data_sources_tenant ON data_sources(tenant_id);
CREATE INDEX idx_insights_tenant_current ON insights(tenant_id, is_current);
CREATE INDEX idx_audit_tenant_timestamp ON audit_logs(tenant_id, timestamp);
```

5.1.2 DuckDB Schema (Analytics)

sql

```
-- Fact Tables (created dynamically based on template)
-- Example: Expenses Table
CREATE TABLE expenses (
    expense_id BIGINT PRIMARY KEY,
    tenant_id VARCHAR NOT NULL,
    source_id VARCHAR NOT NULL,

    -- Mapped columns (strongly typed)
    transaction_date DATE,
    category VARCHAR,
```

```
    description VARCHAR,
    amount DECIMAL(12,2),
    vendor VARCHAR,

    -- Enriched columns
    day_of_week INTEGER,
    month INTEGER,
    quarter INTEGER,
    year INTEGER,
    is_weekend BOOLEAN,

    -- Metadata
    original_row_number INTEGER,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Aggregation Tables (materialized)
CREATE TABLE monthly_expense_summary AS
SELECT
    tenant_id,
    source_id,
    DATE_TRUNC('month', transaction_date) as month,
    category,
    SUM(amount) as total_amount,
    COUNT(*) as transaction_count,
    AVG(amount) as avg_amount,
    MAX(amount) as max_amount,
    MIN(amount) as min_amount
FROM expenses
GROUP BY tenant_id, source_id, month, category;

-- Indexes for performance
CREATE INDEX idx_expenses_tenant_date ON expenses(tenant_id, transaction_date);
CREATE INDEX idx_expenses_category ON expenses(category);
```

5.2 Enterprise Deployment (PostgreSQL)

Full PostgreSQL schema documented in Section 4 of previous conversation - includes:

- Multi-tenant row-level security
- Partitioning by time ranges
- JSONB indexes
- Full-text search

- Materialized views
- PostGIS for location analytics (optional)

5.3 Migration Script

sql

```sql
-- migration/migrate_to_postgresql.sql

-- Enable required extensions
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";
CREATE EXTENSION IF NOT EXISTS "pg_trgm"; -- Fuzzy text search
CREATE EXTENSION IF NOT EXISTS "btree_gin"; -- Multi-column indexes

-- Create schema with same structure as SQLite
-- (Use UUID type instead of TEXT for IDs)

CREATE TABLE tenants (
    tenant_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    company_name VARCHAR(200) NOT NULL,
    industry VARCHAR(100),
    subscription_tier VARCHAR(50),
    encryption_key_hash VARCHAR(255),
    created_at TIMESTAMP DEFAULT NOW(),
    is_active BOOLEAN DEFAULT TRUE
);

-- Enable Row-Level Security
ALTER TABLE tenants ENABLE ROW LEVEL SECURITY;

CREATE POLICY tenant_isolation ON tenants
    USING (tenant_id = current_setting('app.current_tenant')::UUID);

-- ... (continue with all tables)

-- Create partitioned fact tables for better performance
CREATE TABLE expenses (
    expense_id BIGSERIAL,
    tenant_id UUID NOT NULL,
    source_id UUID NOT NULL,
    transaction_date DATE NOT NULL,
    category VARCHAR(100),
    description TEXT,
    amount DECIMAL(12,2),
```

```
    -- Enriched columns
    day_of_week INTEGER,
    month INTEGER,
    quarter INTEGER,
    year INTEGER,
    is_weekend BOOLEAN,

    created_at TIMESTAMP DEFAULT NOW(),

    PRIMARY KEY (tenant_id, transaction_date, expense_id)
) PARTITION BY RANGE (transaction_date);

-- Create partitions
CREATE TABLE expenses_2024 PARTITION OF expenses
    FOR VALUES FROM ('2024-01-01') TO ('2025-01-01');

CREATE TABLE expenses_2025 PARTITION OF expenses
    FOR VALUES FROM ('2025-01-01') TO ('2026-01-01');
```

---

6. Security & Compliance

6.1 Security Architecture

6.1.1 Multi-Layer Security

```
┌─────────────────────────────────────────────────────────┐
│  Layer 1: Network Security                              │
│  • HTTPS/TLS 1.3                                        │
│  • Firewall rules                                       │
│  • Rate limiting                                        │
└─────────────────────────────────────────────────────────┘
                          ↓
┌─────────────────────────────────────────────────────────┐
│  Layer 2: Authentication & Authorization                │
│  • JWT tokens (15 min expiry)                           │
│  • Refresh tokens (7 days)                              │
│  • Role-based access control (RBAC)                     │
│  • Multi-factor authentication (optional)               │
└─────────────────────────────────────────────────────────┘
```

```
                    ↓
┌─────────────────────────────────────────────────┐
│  Layer 3: Application Security                  │
│  • Input validation (Pydantic)                  │
│  • SQL injection prevention (parameterized queries) │
│  • XSS protection                               │
│  • CSRF tokens                                  │
└─────────────────────────────────────────────────┘
                    ↓
┌─────────────────────────────────────────────────┐
│  Layer 4: Data Security                         │
│  • Encryption at rest (AES-256)                 │
│  • Encryption in transit (TLS)                  │
│  • Database encryption (SQLCipher/PostgreSQL TDE) │
│  • Tenant data isolation                        │
└─────────────────────────────────────────────────┘
                    ↓
┌─────────────────────────────────────────────────┐
│  Layer 5: Compliance & Auditing                 │
│  • Complete audit trail                         │
│  • Data retention policies                      │
│  • GDPR compliance features                     │
│  • Regular security scans                       │
└─────────────────────────────────────────────────┘
```

6.1.2 Authentication Implementation

python

```python
# security/auth.py
from datetime import datetime, timedelta
from jose import JWTError, jwt
from passlib.context import CryptContext

class AuthManager:
    SECRET_KEY = os.getenv("SECRET_KEY")
    ALGORITHM = "HS256"
    ACCESS_TOKEN_EXPIRE_MINUTES = 15
    REFRESH_TOKEN_EXPIRE_DAYS = 7

    pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

    @classmethod
```

```python
    def create_access_token(cls, data: dict):
        to_encode = data.copy()
        expire = datetime.utcnow() + timedelta(minutes=cls.ACCESS_TOKEN_EXPIRE_MINUTES)
        to_encode.update({"exp": expire, "type": "access"})
        return jwt.encode(to_encode, cls.SECRET_KEY, algorithm=cls.ALGORITHM)

    @classmethod
    def create_refresh_token(cls, data: dict):
        to_encode = data.copy()
        expire = datetime.utcnow() + timedelta(days=cls.REFRESH_TOKEN_EXPIRE_DAYS)
        to_encode.update({"exp": expire, "type": "refresh"})
        return jwt.encode(to_encode, cls.SECRET_KEY, algorithm=cls.ALGORITHM)

    @classmethod
    def verify_token(cls, token: str):
        try:
            payload = jwt.decode(token, cls.SECRET_KEY, algorithms=[cls.ALGORITHM])
            return payload
        except JWTError:
            return None

    @classmethod
    def hash_password(cls, password: str):
        return cls.pwd_context.hash(password)

    @classmethod
    def verify_password(cls, plain_password: str, hashed_password: str):
        return cls.pwd_context.verify(plain_password, hashed_password)
```

6.1.3 Tenant Isolation

python

```python
# security/tenant_isolation.py
from fastapi import Depends, HTTPException, Request
from sqlalchemy.orm import Session

class TenantIsolation:
    @staticmethod
    async def get_current_tenant(
        request: Request,
        db: Session = Depends(get_db)
    ):
```

```python
        """Extract and validate tenant from token"""
        token = request.headers.get("Authorization", "").replace("Bearer ", "")
        payload = AuthManager.verify_token(token)

        if not payload:
            raise HTTPException(status_code=401, detail="Invalid token")

        tenant_id = payload.get("tenant_id")

        # Validate tenant exists and is active
        tenant = db.query(Tenant).filter(
            Tenant.tenant_id == tenant_id,
            Tenant.is_active == True
        ).first()

        if not tenant:
            raise HTTPException(status_code=403, detail="Invalid tenant")

        # Set tenant context for this request
        request.state.tenant_id = tenant_id
        return tenant_id

    @staticmethod
    def filter_by_tenant(query, model, tenant_id):
        """Automatically add tenant filter to queries"""
        if hasattr(model, 'tenant_id'):
            return query.filter(model.tenant_id == tenant_id)
        return query

# Dependency for protected endpoints
async def require_tenant(
    tenant_id: str = Depends(TenantIsolation.get_current_tenant)
):
    return tenant_id
```

6.2 Data Privacy (GDPR Compliance)

python

```python
# compliance/gdpr.py
class GDPRCompliance:
    def __init__(self, db_session):
        self.db = db_session
```

```python
    def export_user_data(self, user_id: str):
        """Right to data portability (GDPR Article 20)"""
        user = self.db.query(User).filter(User.user_id == user_id).first()

        data_export = {
            'personal_info': {
                'email': user.email,
                'created_at': user.created_at,
                'preferences': user.preferences
            },
            'data_sources': self._get_user_data_sources(user_id),
            'insights': self._get_user_insights(user_id),
            'chat_history': self._get_chat_history(user_id)
        }

        return data_export

    def delete_user_data(self, user_id: str):
        """Right to erasure (GDPR Article 17)"""
        # Anonymize instead of hard delete for audit trail
        user = self.db.query(User).filter(User.user_id == user_id).first()

        user.email = f"deleted_{user_id}@anonymized.com"
        user.password_hash = None
        user.preferences = {}
        user.is_active = False

        # Mark all related data as deleted
        self.db.query(DataSource).filter(
            DataSource.created_by == user_id
        ).update({'is_deleted': True})

        self.db.commit()

    def restrict_processing(self, user_id: str):
        """Right to restriction (GDPR Article 18)"""
        user = self.db.query(User).filter(User.user_id == user_id).first()
        user.data_processing_restricted = True
        self.db.commit()
```

6.3 Audit Logging

python

```python
# security/audit.py
from enum import Enum

class AuditAction(Enum):
    LOGIN = "login"
    LOGOUT = "logout"
    DATA_UPLOAD = "data_upload"
    DATA_VIEW = "data_view"
    INSIGHT_GENERATE = "insight_generate"
    REPORT_DOWNLOAD = "report_download"
    TEMPLATE_MODIFY = "template_modify"
    USER_CREATE = "user_create"
    USER_DELETE = "user_delete"

class AuditLogger:
    def __init__(self, db_session):
        self.db = db_session

    def log(
        self,
        tenant_id: str,
        user_id: str,
        action: AuditAction,
        resource_type: str,
        resource_id: str,
        metadata: dict = None
    ):
        audit_entry = AuditLog(
            tenant_id=tenant_id,
            user_id=user_id,
            action_type=action.value,
            resource_type=resource_type,
            resource_id=resource_id,
            metadata=json.dumps(metadata) if metadata else None,
            timestamp=datetime.utcnow()
        )

        self.db.add(audit_entry)
        self.db.commit()

    def get_user_activity(self, user_id: str, days: int = 30):
```

```
        """Get user activity for last N days"""
        cutoff = datetime.utcnow() - timedelta(days=days)

        return self.db.query(AuditLog).filter(
            AuditLog.user_id == user_id,
            AuditLog.timestamp >= cutoff
        ).order_by(AuditLog.timestamp.desc()).all()
```

---

7. Core Features

7.1 Feature List

7.1.1 Data Management Features

Feature	Description	Priority
File Upload	CSV, JSON, TXT support with drag-drop	P0
Schema Detection	Automatic column type inference	P0
Template Selection	Industry-based template matching	P0
Smart Mapping	4-layer intelligent column mapping	P0
Data Validation	Type checking, range validation, business rules	P0
Multi-Source Support	Link related data sources	P1
Scheduled Imports	Automated daily/weekly data refresh	P2
API Integration	Connect to external APIs	P2

7.1.2 Analytics Features

Feature	Description	Priority
Predefined Reports	Industry-specific dashboards	P0
Natural Language Insights	Executive summaries in plain English	P0
AI Chatbot	Ask questions in natural language	P0
Trend Analysis	Time series forecasting	P0
Anomaly Detection	Automatic outlier identification	P1
Comparative Analytics	Period-over-period comparisons	P1
Predictive Models	ML-based forecasting	P2
What-If Scenarios	Simulate business scenarios	P2

7.1.3 Collaboration Features

```
| Feature | Description | Priority |
|---------|-------------|----------|
| **User Management** | Multi-user support with roles | P0 |
| **Shared Dashboards** | Share reports with team | P1 |
| **Comments/Annotations** | Discuss insights inline | P2 |
| **Scheduled Reports** | Email reports automatically | P1 |
| **Export Options** | PDF, PPTX, Excel exports | P1 |

### 7.2 User Workflows

#### 7.2.1 First-Time Setup (Day 1)
```
User Journey: Small Business Owner - Day 1
═══

1. Sign Up
   └─> Choose industry: "Retail - E-commerce"
   └─> System loads relevant templates

2. Upload First Data Source
   └─> Drag expense CSV file
   └─> System detects: date, category, amount, vendor
   └─> Auto-maps to "Expense Tracking" template
   └─> Confidence: 95%

3. Review Mapping
   └─> System shows mapping table
   └─> User confirms or adjusts
   └─> Click "Process Data"

4. Instant Insights (< 30 seconds)
   └─> See 3 predefined reports
       • Monthly Expense Summary
       • Category Breakdown
       • Trend Analysis
   └─> Read 5 natural language insights
       • "Your dining expenses increased 40% in January"
       • "Weekends account for 60% of total spend"
   └─> Ask chatbot: "What's my average daily spend?"

5. Next Steps Suggested
   └─> "Upload revenue data to see profit analysis"
   └─> "Add budget data for variance tracking"
```
```

```
Total Time: 5 minutes
Insights Generated: 8
```

7.2.2 Power User Workflow

```
User Journey: Data Analyst - Monthly Reporting
═══════════════════════════════════════════════

1. Scheduled Data Refresh
   └─> System auto-imports monthly sales data
   └─> Validates against last month's schema
   └─> Sends notification if anomalies detected

2. Review Auto-Generated Insights
   └─> 15 new insights waiting
   └─> Ranked by business impact
   └─> Drill down into top 3

3. Custom Analysis via Chatbot
   └─> "Compare Q1 2025 vs Q1 2024 by product category"
   └─> "Show me transactions over $10,000"
   └─> "Which customers haven't purchased in 90 days?"

4. Create Executive Report
   └─> Select template: "Monthly Business Review"
   └─> Customize KPIs
   └─> Add custom commentary
   └─> Export as PowerPoint

5. Schedule Distribution
   └─> Email report to executives every 1st of month
   └─> Slack notification to #finance channel

Total Time: 20 minutes/month
Manual work eliminated: 4 hours
```

---

8. AI/ML Capabilities

8.1 Machine Learning Models

8.1.1 Model Catalog

Model Type	Use Case	Algorithm	Update Frequency
Time Series Forecast	Revenue/demand prediction	Prophet	Monthly
Anomaly Detection	Fraud/outlier detection	Isolation Forest	Real-time
Clustering	Customer segmentation	K-Means	Quarterly
Classification	Transaction categorization	Random Forest	As needed
Text Analysis	Sentiment/topic extraction	BERT	Per query
Regression	Sales driver analysis	XGBoost	Monthly

8.1.2 Model Training Pipeline

python

```python
# ml/model_trainer.py
class ModelTrainingPipeline:
    def __init__(self, tenant_id, source_id):
        self.tenant_id = tenant_id
        self.source_id = source_id
        self.model_registry = ModelRegistry()

    def train_forecasting_model(self, df, date_col, target_col):
        """Train Prophet model for forecasting"""

        # Prepare data
        train_data = df[[date_col, target_col]].rename(
            columns={date_col: 'ds', target_col: 'y'}
        )

        # Split train/test
        split_date = train_data['ds'].max() - pd.DateOffset(days=30)
        train = train_data[train_data['ds'] <= split_date]
        test = train_data[train_data['ds'] > split_date]

        # Train model
        model = Prophet(
            yearly_seasonality=True,
            weekly_seasonality=True,
            changepoint_prior_scale=0.05
        )
```

```
        model.fit(train)

        # Evaluate
        forecast = model.predict(test[['ds']])
        mape = self._calculate_mape(test['y'], forecast['yhat'])

        # Save model
        model_path = f"models/{self.tenant_id}/forecast_{target_col}.pkl"
        joblib.dump(model, model_path)

        # Register in database
        self.model_registry.register(
            tenant_id=self.tenant_id,
            model_name=f"forecast_{target_col}",
            model_type="prophet",
            model_path=model_path,
            metrics={'mape': mape},
            training_data_size=len(train)
        )

        return model, mape

    def train_anomaly_detector(self, df, feature_cols):
        """Train Isolation Forest for anomaly detection"""

        X = df[feature_cols].fillna(df[feature_cols].median())

        model = IsolationForest(
            contamination=0.05,
            random_state=42,
            n_estimators=100
        )
        model.fit(X)

        # Validate
        predictions = model.predict(X)
        anomaly_rate = (predictions == -1).sum() / len(X)

        # Save
        model_path = f"models/{self.tenant_id}/anomaly_detector.pkl"
        joblib.dump(model, model_path)

        self.model_registry.register(
```

```
            tenant_id=self.tenant_id,
            model_name="anomaly_detector",
            model_type="isolation_forest",
            model_path=model_path,
            metrics={'anomaly_rate': anomaly_rate}
        )

        return model
```

8.2 Natural Language Processing

8.2.1 Chatbot Architecture (RAG-based)

python

```
# chatbot/rag_engine.py
from langchain.vectorstores import Chroma
from langchain.embeddings import SentenceTransformerEmbeddings
from langchain.text_splitter import RecursiveCharacterTextSplitter

class RAGChatBot:
    def __init__(self, tenant_id):
        self.tenant_id = tenant_id
        self.embeddings = SentenceTransformerEmbeddings(
            model_name="all-MiniLM-L6-v2"
        )
        self.vector_store = self._load_vector_store()
        self.llm = AnthropicClient()

    def _load_vector_store(self):
        """Load pre-computed insights into vector database"""

        # Get all insights for this tenant
        insights = db.query(Insight).filter(
            Insight.tenant_id == self.tenant_id,
            Insight.is_current == True
        ).all()

        # Create documents
        documents = []
        for insight in insights:
            doc_text = f"""
            Title: {insight.title}
            Type: {insight.insight_type}
```

```
            Category: {insight.category}
            Description: {insight.description}
            Narrative: {insight.narrative}
            Period: {insight.data_period_start} to {insight.data_period_end}
            """
            documents.append({
                'text': doc_text,
                'metadata': {
                    'insight_id': insight.insight_id,
                    'type': insight.insight_type,
                    'importance': insight.importance_score
                }
            })

        # Create vector store
        vector_store = Chroma.from_texts(
            texts=[d['text'] for d in documents],
            metadatas=[d['metadata'] for d in documents],
            embedding=self.embeddings,
            persist_directory=f"vector_db/{self.tenant_id}"
        )

        return vector_store

    def answer_question(self, question: str, conversation_history: list = None):
        """Answer user question using RAG"""

        # 1. Retrieve relevant insights
        relevant_docs = self.vector_store.similarity_search(
            question,
            k=3
        )

        context = "\n\n".join([doc.page_content for doc in relevant_docs])

        # 2. Determine if SQL query is needed
        intent = self._classify_intent(question)

        if intent == "needs_sql":
            # Generate and execute SQL
            sql = self._generate_sql(question)
            query_results = self._execute_safe_sql(sql)
            context += f"\n\nQuery Results:\n{query_results}"
```

```
        # 3. Generate response with LLM
        prompt = f"""
You are a business analytics assistant.

Context from past insights:
{context}

Conversation history:
{json.dumps(conversation_history or [], indent=2)}

User question: {question}

Provide a clear, concise answer. Include specific numbers and cite insights when relevant.
If you don't have enough information, say so and suggest what data would be helpful.
"""

        response = self.llm.complete(prompt, max_tokens=500)

        # 4. Add visualization if appropriate
        if self._should_visualize(question):
            chart_data = self._prepare_chart_data(question, context)
            return {
                'text': response,
                'chart': chart_data
            }

        return {'text': response}

    def _generate_sql(self, question: str):
        """Use LLM to convert natural language to SQL"""

        schema = self._get_database_schema()

        prompt = f"""
Given this database schema:
{schema}

Convert this question to SQL:
"{question}"

Rules:
- Return only the SQL query
```

```
- Use proper table aliases
- Limit results to 1000 rows
- Only use SELECT statements (no INSERT/UPDATE/DELETE)
"""

        sql = self.llm.complete(prompt, max_tokens=300)

        # Validate SQL is safe
        if not self._is_safe_sql(sql):
            raise ValueError("Generated SQL failed safety check")

        return sql

    def _is_safe_sql(self, sql: str):
        """Validate SQL doesn't contain dangerous operations"""

        dangerous_keywords = [
            'DROP', 'DELETE', 'UPDATE', 'INSERT', 'ALTER',
            'CREATE', 'TRUNCATE', 'EXEC', 'EXECUTE'
        ]

        sql_upper = sql.upper()
        return not any(keyword in sql_upper for keyword in dangerous_keywords)
```

8.3 Semantic Understanding (Embeddings)

python

```
# ml/semantic_matcher.py
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity

class SemanticMatcher:
    def __init__(self):
        self.model = SentenceTransformer('all-MiniLM-L6-v2')

    def find_similar_columns(
        self,
        source_columns: list,
        template_columns: list,
        threshold: float = 0.7
    ):
        """Find semantically similar columns using embeddings"""
```

```
        # Generate embeddings
        source_embeddings = self.model.encode(source_columns)
        template_embeddings = self.model.encode(template_columns)

        # Calculate similarities
        similarities = cosine_similarity(source_embeddings, template_embeddings)

        # Find best matches
        matches = {}
        for i, source_col in enumerate(source_columns):
            best_match_idx = similarities[i].argmax()
            best_score = similarities[i][best_match_idx]

            if best_score >= threshold:
                matches[source_col] = {
                    'template_column': template_columns[best_match_idx],
                    'confidence': float(best_score),
                    'method': 'semantic'
                }

        return matches
```

---

9. Development Plan

9.1 Project Phases

```
Timeline: 6 Months to MVP + 3 Months to V1.0
═══════════════════════════════════════════

Phase 0: Setup & Foundation (Weeks 1-2)
├── Development environment setup
├── Technology stack finalization
├── Database design
├── Project structure
└── CI/CD pipeline

Phase 1: Core Data Pipeline (Weeks 3-6)
├── File upload & parsing
├── Schema detection
```

```
├── Template management system
├── Column mapping engine (Layer 1-2)
├── Basic data validation
└── SQLite + DuckDB setup

Phase 2: Analytics Engine (Weeks 7-10)
├── Statistical analyzers
├── Time series analysis
├── Anomaly detection
├── Insight generation
└── Insight ranking

Phase 3: Output Generation (Weeks 11-14)
├── Predefined report templates
├── Natural language generation
├── Basic chatbot (no RAG yet)
├── Export functionality (PDF, Excel)
└── API endpoints

Phase 4: UI Development (Weeks 15-18)
├── React frontend setup
├── Dashboard views
├── Data upload interface
├── Report viewer
├── Chat interface
└── User authentication UI

Phase 5: AI Enhancement (Weeks 19-22)
├── Advanced mapping (Layer 3-4)
├── RAG-based chatbot
├── SQL generation
├── Predictive models
└── Template learning

Phase 6: Enterprise Features (Weeks 23-26)
├── PostgreSQL migration tool
├── Multi-tenant isolation
├── Advanced security
├── Audit logging
└── Performance optimization

Phase 7: Testing & Refinement (Weeks 27-30)
├── Comprehensive testing
```

```
├── Bug fixes
├── Performance tuning
├── Documentation
└── User acceptance testing

Phase 8: Launch Preparation (Weeks 31-36)
├── Production deployment
├── Monitoring setup
├── Customer onboarding tools
├── Training materials
└── Marketing website
```

9.2 Sprint Planning (2-Week Sprints)

Sprint 1-2: Foundation

Sprint 1 Goals:
- [ ] Setup development environment
- [ ] Initialize Git repository
- [ ] Create project structure
- [ ] Setup FastAPI skeleton
- [ ] Design database schema (SQLite)
- [ ] Create initial models with SQLAlchemy

Sprint 1 Deliverables:
- Working local development environment
- Basic API with health check endpoint
- Database tables created
- Authentication skeleton

Sprint 2 Goals:
- [ ] File upload endpoint
- [ ] CSV/JSON parsing
- [ ] Basic schema detection
- [ ] Template model creation
- [ ] First industry template (retail expenses)

Sprint 2 Deliverables:
- API can accept file uploads
- Files are parsed and schema detected
- Template stored in database

Sprint 3-4: Intelligent Mapping

Sprint 3 Goals:
- [ ] Implement exact name matching
- [ ] Implement semantic similarity matching
- [ ] Create mapping confidence scoring
- [ ] Build mapping review UI (basic)
- [ ] Manual mapping adjustment

Sprint 3 Deliverables:
- 2-layer mapping working
- User can review/adjust mappings
- Mapping results saved to database

Sprint 4 Goals:
- [ ] Content-based column detection
- [ ] Historical mapping learning
- [ ] Mapping accuracy metrics
- [ ] Template versioning

Sprint 4 Deliverables:
- 4-layer mapping complete
- Mapping improves with usage
- Template evolution tracking

Sprint 5-6: Data Validation & Storage

Sprint 5 Goals:
- [ ] Type validation
- [ ] Range validation
- [ ] Business rule validation
- [ ] Data enrichment (derived columns)
- [ ] DuckDB integration

Sprint 5 Deliverables:
- Validated data stored in DuckDB
- Enriched columns auto-generated
- Validation errors reported to user

Sprint 6 Goals:
- [ ] Multi-source support
- [ ] Relationship detection
- [ ] Data quality scoring

- [ ] Automated data profiling

Sprint 6 Deliverables:
- Can upload multiple related sources
- Relationships auto-detected
- Data quality metrics shown

Sprint 7-8: Analytics Engine

Sprint 7 Goals:
- [ ] Statistical analyzer
- [ ] Time series analyzer (Prophet)
- [ ] Basic insights generation
- [ ] Insight storage

Sprint 7 Deliverables:
- Statistical insights generated
- Time series trends detected
- Insights stored with confidence scores

Sprint 8 Goals:
- [ ] Anomaly detector (Isolation Forest)
- [ ] Comparative analyzer
- [ ] Insight ranking algorithm
- [ ] Insight caching

Sprint 8 Deliverables:
- Anomalies auto-detected
- Insights ranked by importance
- Fast insight retrieval

Sprint 9-10: Report Generation

Sprint 9 Goals:
- [ ] Report template system
- [ ] KPI calculation engine
- [ ] Chart data generation
- [ ] HTML report renderer

Sprint 9 Deliverables:
- 3 predefined report templates
- Reports generated from data
- HTML export working

Sprint 10 Goals:
- [ ] PDF export (WeasyPrint)
- [ ] PowerPoint export (python-pptx)
- [ ] Excel export
- [ ] Report scheduling

Sprint 10 Deliverables:
- Multi-format export working
- Scheduled reports functional

Sprint 11-12: Natural Language Features

Sprint 11 Goals:
- [ ] LLM integration (Anthropic Claude)
- [ ] Natural language insight generation
- [ ] Executive summary creation
- [ ] Insight explanation generation

Sprint 11 Deliverables:
- Insights converted to natural language
- Executive summaries auto-generated

Sprint 12 Goals:
- [ ] Basic chatbot
- [ ] Intent classification
- [ ] Simple Q&A from insights
- [ ] Conversation history

Sprint 12 Deliverables:
- Chat interface working
- Can answer basic questions
- Conversation persisted

Sprint 13-15: Frontend Development

Sprint 13 Goals:
- [ ] React project setup
- [ ] Authentication UI
- [ ] Dashboard layout
- [ ] File upload UI
- [ ] Navigation structure

```
**Sprint 13 Deliverables:**
- User can login
- Dashboard skeleton visible
- File upload working

**Sprint 14 Goals:**
- [ ] Data mapping UI
- [ ] Insight viewer
- [ ] Report viewer
- [ ] Chart components

**Sprint 14 Deliverables:**
- Full upload → insights workflow in UI
- Reports displayed nicely
- Interactive charts

**Sprint 15 Goals:**
- [ ] Chat interface
- [ ] Settings page
- [ ] User management UI
- [ ] Responsive design

**Sprint 15 Deliverables:**
- Complete frontend MVP
- Mobile-friendly
- All core features accessible

#### Sprint 16-17: Advanced AI

**Sprint 16 Goals:**
- [ ] Vector database setup (Chroma)
- [ ] RAG implementation
- [ ] SQL generation from natural language
- [ ] Safe SQL execution

**Sprint 16 Deliverables:**
- RAG-based chatbot working
- Can answer ad-hoc analytical questions
- SQL queries generated safely

**Sprint 17 Goals:**
- [ ] Predictive models
- [ ] ML model registry
```

```
- [ ] Model retraining pipeline
- [ ] Model performance tracking

**Sprint 17 Deliverables:**
- Forecasting working
- Models auto-retrain monthly
- Model accuracy tracked

#### Sprint 18-19: Enterprise Features

**Sprint 18 Goals:**
- [ ] PostgreSQL migration tool
- [ ] Row-level security implementation
- [ ] Multi-tenant testing
- [ ] Data encryption

**Sprint 18 Deliverables:**
- Can migrate SQLite → PostgreSQL
- Complete tenant isolation
- Encrypted data at rest

**Sprint 19 Goals:**
- [ ] Audit logging
- [ ] GDPR compliance features
- [ ] Advanced backup/restore
- [ ] Performance optimization

**Sprint 19 Deliverables:**
- Complete audit trail
- GDPR data export/deletion
- Optimized queries

#### Sprint 20-22: Testing & Polish

**Sprint 20 Goals:**
- [ ] Unit test coverage >80%
- [ ] Integration tests
- [ ] Performance testing
- [ ] Security audit

**Sprint 20 Deliverables:**
- Comprehensive test suite
- Performance benchmarks
```

- Security vulnerabilities fixed

Sprint 21 Goals:
- [ ] User acceptance testing
- [ ] Bug fixes
- [ ] Documentation
- [ ] Deployment guides

Sprint 21 Deliverables:
- Stable, tested application
- Complete documentation
- Deployment runbooks

Sprint 22 Goals:
- [ ] Production deployment
- [ ] Monitoring setup
- [ ] Backup automation
- [ ] Customer onboarding materials

Sprint 22 Deliverables:
- Application in production
- Monitoring dashboards
- Customer-ready

9.3 Team Structure

Recommended Team (MVP Phase)
```
Project Manager (1)
├── Sprint planning
├── Stakeholder communication
└── Risk management

Technical Lead / Architect (1)
├── Technical decisions
├── Code reviews
└── Architecture guidance

Backend Developers (2)
├── API development
├── Database design
├── ML integration
└── Testing
```

```
Frontend Developer (1)
├── React development
├── UI/UX implementation
└── Component library

ML Engineer (1 - Part-time)
├── Model development
├── ML pipeline
└── Model optimization

DevOps Engineer (1 - Part-time)
├── CI/CD
├── Deployment
└── Monitoring

QA Engineer (1)
├── Test planning
├── Testing execution
└── Bug tracking

Total: 7-8 people
```

9.4 Technology Decisions

Decision Point	Options Considered	Selected	Rationale
Backend Framework	Django, Flask, FastAPI	**FastAPI**	Modern, async, auto API docs
ORM	Raw SQL, SQLAlchemy, Tortoise	**SQLAlchemy**	Database agnostic, mature
Small DB	SQLite, DuckDB, Embedded Postgres	**SQLite + DuckDB**	Zero config, DuckDB for analytics
Enterprise DB	PostgreSQL, MySQL, MongoDB	**PostgreSQL**	JSON support, full-featured
Frontend	React, Vue, Angular	**React + Next.js**	Ecosystem, SSR capability
LLM	OpenAI, Anthropic, Open-source	**Anthropic Claude**	Strong reasoning, tool use

Decision Point	Options Considered	Selected	Rationale
ML Library	scikit-learn, PyTorch, TensorFlow	**scikit-learn**	Sufficient for classical ML
Time Series	statsmodels, Prophet, NeuralProphet	**Prophet**	Easy, robust, good defaults
Vector DB	Pinecone, Chroma, Weaviate	**Chroma**	Embedded option, simple
Task Queue	Celery, RQ, Dramatiq	**Celery**	Mature, well-documented
Testing	unittest, pytest, nose	**pytest**	Modern, fixture support

10. Deployment Strategy

10.1 Deployment Tiers

10.1.1 Small Deployment (Self-Hosted)

Target: 1-25 users, <50GB data

yaml

```yaml
# docker-compose.yml (Small Deployment)
version: '3.8'

services:
  app:
    build: .
    container_name: ai-analytics-app
    ports:
      - "8000:8000"
    volumes:
      - ./data:/app/data  # SQLite/DuckDB files
      - ./uploads:/app/uploads
      - ./backups:/app/backups
    environment:
      - DEPLOYMENT_TYPE=small
      - SECRET_KEY=${SECRET_KEY}
      - ANTHROPIC_API_KEY=${ANTHROPIC_API_KEY}
    restart: unless-stopped

  nginx:
    image: nginx:alpine
    ports:
```

```
      - "80:80"
      - "443:443"
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf
      - ./ssl:/etc/nginx/ssl
    depends_on:
      - app
    restart: unless-stopped
```

Setup Instructions:

```bash
# 1. Clone repository
git clone https://github.com/yourcompany/ai-analytics.git
cd ai-analytics

# 2. Configure environment
cp .env.example .env
# Edit .env with your settings

# 3. Start application
docker-compose up -d

# 4. Access application
# http://localhost (or your domain)
```

10.1.2 Enterprise Deployment (Cloud)

Target: 100+ users, 100GB+ data

```yaml
# docker-compose.enterprise.yml
version: '3.8'

services:
  app:
    build: .
    deploy:
      replicas: 3
      restart_policy:
        condition: on-failure
    environment:
      - DEPLOYMENT_TYPE=enterprise
```

```
      - DATABASE_URL=${DATABASE_URL}
      - REDIS_URL=${REDIS_URL}
      - S3_BUCKET=${S3_BUCKET}

  postgres:
    image: postgres:15
    volumes:
      - postgres_data:/var/lib/postgresql/data
    environment:
      - POSTGRES_DB=analytics
      - POSTGRES_USER=${DB_USER}
      - POSTGRES_PASSWORD=${DB_PASSWORD}

  redis:
    image: redis:7-alpine
    volumes:
      - redis_data:/data

  celery_worker:
    build: .
    command: celery -A app.celery worker --loglevel=info
    deploy:
      replicas: 2
    depends_on:
      - redis
      - postgres

  celery_beat:
    build: .
    command: celery -A app.celery beat --loglevel=info
    depends_on:
      - redis
      - postgres

volumes:
  postgres_data:
  redis_data:
```

10.2 Scaling Strategy

```
Small → Medium → Enterprise
═══════════════════════════
```

```
SMALL (0-25 users)
├── SQLite + DuckDB
├── Single server
├── No load balancer
└── File-based storage

MEDIUM (25-100 users)
├── PostgreSQL (managed)
├── 2-3 app servers
├── Load balancer
├── S3 for files
└── Redis cache

ENTERPRISE (100+ users)
├── PostgreSQL (HA cluster)
├── Auto-scaling app servers (3-10)
├── CDN + load balancer
├── Distributed file storage
├── Redis cluster
├── Separate analytics DB
└── Vector DB cluster
```

10.3 Migration Procedure

SQLite → PostgreSQL Migration

bash

```
# Step 1: Backup current data
python scripts/backup_sqlite.py

# Step 2: Setup PostgreSQL
# Create database and user

# Step 3: Run migration
python scripts/migrate_to_postgres.py \
    --source sqlite:///data/app_metadata.db \
    --target postgresql://user:pass@localhost/analytics

# Step 4: Verify data integrity
python scripts/verify_migration.py

# Step 5: Update configuration
```

```
# Change DEPLOYMENT_TYPE=enterprise in .env

# Step 6: Restart application
docker-compose down
docker-compose -f docker-compose.enterprise.yml up -d

# Step 7: Monitor for issues
tail -f logs/app.log
```

10.4 Infrastructure as Code
Terraform for AWS Deployment

hcl

```
# infrastructure/main.tf

# VPC
module "vpc" {
  source = "terraform-aws-modules/vpc/aws"

  name = "ai-analytics-vpc"
  cidr = "10.0.0.0/16"

  azs             = ["us-east-1a", "us-east-1b"]
  private_subnets = ["10.0.1.0/24", "10.0.2.0/24"]
  public_subnets  = ["10.0.101.0/24", "10.0.102.0/24"]

  enable_nat_gateway = true
}

# RDS PostgreSQL
resource "aws_db_instance" "postgres" {
  identifier = "ai-analytics-db"
  engine     = "postgres"
  engine_version = "15.4"
  instance_class = "db.t3.medium"

  allocated_storage = 100
  storage_encrypted = true

  db_name  = "analytics"
  username = var.db_username
  password = var.db_password
```

```
  vpc_security_group_ids = [aws_security_group.db.id]
  db_subnet_group_name   = aws_db_subnet_group.main.name

  backup_retention_period = 7
  backup_window          = "03:00-04:00"
  maintenance_window     = "mon:04:00-mon:05:00"

  multi_az = true
}

# ElastiCache Redis
resource "aws_elasticache_cluster" "redis" {
  cluster_id           = "ai-analytics-cache"
  engine               = "redis"
  node_type            = "cache.t3.micro"
  num_cache_nodes      = 1
  parameter_group_name = "default.redis7"
  port                 = 6379

  subnet_group_name    = aws_elasticache_subnet_group.main.name
  security_group_ids   = [aws_security_group.cache.id]
}

# ECS Cluster for App
resource "aws_ecs_cluster" "main" {
  name = "ai-analytics-cluster"
}

# Application Load Balancer
resource "aws_lb" "main" {
  name               = "ai-analytics-alb"
  internal           = false
  load_balancer_type = "application"
  security_groups    = [aws_security_group.alb.id]
  subnets            = module.vpc.public_subnets
}

# S3 Bucket for Files
resource "aws_s3_bucket" "uploads" {
  bucket = "ai-analytics-uploads-${var.environment}"

  server_side_encryption_configuration {
```

```
    rule {
      apply_server_side_encryption_by_default {
        sse_algorithm = "AES256"
      }
    }
  }

  versioning {
    enabled = true
  }
}
```

---

11. Testing Strategy

11.1 Testing Pyramid

```
                    ▲
                   /E\      E2E Tests (10%)
                  /───\     • User workflows
                 /  M  \    • Integration tests
                /───────\
               /    U    \  Unit Tests (70%)
              /─────────────\ • Function tests
             ─────────────────• Component tests

Manual Testing (20%)
- Exploratory testing
- Usability testing
- Security testing
```

11.2 Test Coverage Requirements

Component	Target Coverage	Test Types
Core Business Logic	90%+	Unit, Integration
API Endpoints	85%+	Integration, E2E
Data Pipeline	90%+	Unit, Integration

Component	Target Coverage	Test Types
ML Models	80%+	Unit, Performance
Database Operations	85%+	Integration
Frontend Components	75%+	Unit, E2E

11.3 Test Examples

11.3.1 Unit Tests

python

```python
# tests/test_mapping.py
import pytest
from app.mapping import IntelligentMapper
from app.models import IndustryTemplate

class TestIntelligentMapper:
    @pytest.fixture
    def sample_dataframe(self):
        return pd.DataFrame({
            'date': ['2024-01-01', '2024-01-02'],
            'amt': [100.50, 200.75],
            'cat': ['food', 'travel']
        })

    @pytest.fixture
    def sample_template(self):
        return IndustryTemplate(
            columns={
                'transaction_date': {'aliases': ['date', 'txn_date']},
                'amount': {'aliases': ['amt', 'total']},
                'category': {'aliases': ['cat', 'type']}
            }
        )

    def test_exact_match_mapping(self, sample_dataframe, sample_template):
        mapper = IntelligentMapper()
        mappings = mapper.map_columns(sample_dataframe, sample_template)

        assert mappings['date']['template_column'] == 'transaction_date'
        assert mappings['date']['confidence'] == 1.0
```

```
        assert mappings['date']['method'] == 'exact'

    def test_semantic_similarity_mapping(self):
        mapper = IntelligentMapper()

        source_cols = ['total_price', 'customer_name']
        template_cols = ['amount', 'customer']

        matches = mapper._semantic_match(source_cols, template_cols)

        assert 'total_price' in matches
        assert matches['total_price']['confidence'] > 0.7

    def test_content_based_detection(self):
        df = pd.DataFrame({
            'col1': ['john@example.com', 'jane@example.com'],
            'col2': ['$100.50', '$200.75']
        })

        mapper = IntelligentMapper()
        detections = mapper._analyze_column_content(df)

        assert detections['col1'] == 'email_address'
        assert detections['col2'] == 'currency_amount'
```

11.3.2 Integration Tests

python

```
# tests/test_data_pipeline.py
import pytest
from fastapi.testclient import TestClient
from app.main import app

client = TestClient(app)

class TestDataPipeline:
    def test_complete_upload_to_insights_workflow(self):
        # Step 1: Upload file
        with open('tests/fixtures/sample_expenses.csv', 'rb') as f:
            response = client.post(
                "/api/v1/data-sources/upload",
                files={"file": ("expenses.csv", f, "text/csv")},
```

```
            headers={"Authorization": f"Bearer {self.auth_token}"}
        )

        assert response.status_code == 200
        source_id = response.json()['source_id']

        # Step 2: Wait for processing
        import time
        time.sleep(5)

        # Step 3: Check insights generated
        response = client.get(
            f"/api/v1/insights?source_id={source_id}",
            headers={"Authorization": f"Bearer {self.auth_token}"}
        )

        assert response.status_code == 200
        insights = response.json()
        assert len(insights) > 0
        assert any(i['insight_type'] == 'statistical' for i in insights)
```

11.3.3 Performance Tests

python

```
# tests/test_performance.py
import pytest
from app.analytics import InsightEngine

class TestPerformance:
    @pytest.mark.slow
    def test_insight_generation_performance(self):
        """Ensure insights generated in <5 seconds for 10K rows"""

        # Create large dataset
        df = pd.DataFrame({
            'date': pd.date_range('2023-01-01', periods=10000),
            'amount': np.random.uniform(10, 1000, 10000),
            'category': np.random.choice(['A', 'B', 'C'], 10000)
        })

        engine = InsightEngine()
```

```
        import time
        start = time.time()
        insights = engine.generate_insights(df, template, mapping)
        duration = time.time() - start

        assert duration < 5.0, f"Took {duration}s, expected <5s"
        assert len(insights) > 0
```

11.4 CI/CD Pipeline

yaml

```yaml
# .github/workflows/ci.yml
name: CI/CD Pipeline

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest

    services:
      postgres:
        image: postgres:15
        env:
          POSTGRES_PASSWORD: postgres
        options: >-
          --health-cmd pg_isready
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5

    steps:
    - uses: actions/checkout@v3

    - name: Set up Python
      uses: actions/setup-python@v4
      with:
        python-version: '3.11'
```

```yaml
    - name: Install dependencies
      run: |
        pip install -r requirements.txt
        pip install -r requirements-dev.txt

    - name: Run linters
      run: |
        black --check .
        flake8 .
        mypy .

    - name: Run tests
      env:
        DATABASE_URL: postgresql://postgres:postgres@localhost/test_db
      run: |
        pytest --cov=app --cov-report=xml --cov-report=html

    - name: Upload coverage
      uses: codecov/codecov-action@v3
      with:
        file: ./coverage.xml

    - name: Build Docker image
      run: docker build -t ai-analytics:${{ github.sha }} .

    - name: Security scan
      run: |
        pip install safety bandit
        safety check
        bandit -r app/

  deploy:
    needs: test
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main'

    steps:
    - name: Deploy to production
      # Deployment steps here
      run: echo "Deploying to production..."
```

12. Maintenance & Support

12.1 Monitoring & Alerting

python

```python
# monitoring/metrics.py
from prometheus_client import Counter, Histogram, Gauge

# Metrics
upload_counter = Counter(
    'data_uploads_total',
    'Total number of data uploads',
    ['tenant_id', 'status']
)

insight_generation_time = Histogram(
    'insight_generation_seconds',
    'Time to generate insights',
    ['insight_type']
)

active_users = Gauge(
    'active_users',
    'Number of active users',
    ['tenant_id']
)

# Usage
upload_counter.labels(tenant_id='123', status='success').inc()
insight_generation_time.labels(insight_type='statistical').observe(2.5)
```

Grafana Dashboard Metrics:

- Request rate and latency
- Error rates by endpoint
- Database query performance
- Insight generation time
- User activity
- System resource usage (CPU, memory, disk)

12.2 Logging Strategy

python

```python
# logging_config.py
import logging
from pythonjsonlogger import jsonlogger
```

```
class CustomJsonFormatter(jsonlogger.JsonFormatter):
    def add_fields(self, log_record, record, message_dict):
        super().add_fields(log_record, record, message_dict)
        log_record['tenant_id'] = getattr(record, 'tenant_id', None)
        log_record['user_id'] = getattr(record, 'user_id', None)
        log_record['request_id'] = getattr(record, 'request_id', None)

# Configure logging
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s %(levelname)s %(message)s'
)

logger = logging.getLogger('ai_analytics')
handler = logging.StreamHandler()
handler.setFormatter(CustomJsonFormatter())
logger.addHandler(handler)
```

12.3 Backup Strategy

Automated Backups:

python

```
# backup/scheduler.py
from apscheduler.schedulers.background import BackgroundScheduler

scheduler = BackgroundScheduler()

# Daily database backup at 2 AM
scheduler.add_job(
    backup_manager.create_backup,
    'cron',
    hour=2,
    id='daily_backup'
)

# Weekly full backup to S3 (Sunday 3 AM)
scheduler.add_job(
    backup_manager.backup_to_s3,
    'cron',
    day_of_week='sun',
    hour=3,
```

```
    id='weekly_s3_backup'
)

scheduler.start()
```

Backup Retention Policy:

- Daily backups: Keep 7 days
- Weekly backups: Keep 4 weeks
- Monthly backups: Keep 12 months

12.4 Support Tiers

Tier	Response Time	Channels	Availability
Community	Best effort	Forum, GitHub	-
Standard	24 hours	Email	Business hours
Premium	4 hours	Email, Chat	24/7
Enterprise	1 hour	Email, Chat, Phone	24/7 + dedicated manager

12.5 Update Strategy

Version Numbering: MAJOR.MINOR.PATCH (e.g., 1.2.3)

Release Cadence:

- Major releases: Quarterly (breaking changes)
- Minor releases: Monthly (new features)
- Patch releases: As needed (bug fixes)

Update Process:

bash

```
# 1. Announcement (1 week before)
# 2. Backup current system
python scripts/backup_all.py

# 3. Update code
git pull origin main

# 4. Run migrations
python scripts/migrate_database.py

# 5. Restart services
docker-compose down
docker-compose up -d
```

```
# 6. Verify
python scripts/health_check.py
```

---

13. Success Metrics

13.1 Product Metrics

Key Performance Indicators (KPIs):

Metric	Target	Measurement
Time to First Insight	< 5 minutes	User uploads data → sees first insight
Mapping Accuracy	> 90%	% of correct auto-mappings
Insight Quality Score	> 4.0/5.0	User ratings on insights
Chat Response Accuracy	> 85%	Correct answers to queries
User Retention (30-day)	> 60%	Active users after 30 days
Daily Active Users / MAU	> 40%	Engagement metric

13.2 Technical Metrics

Metric	Target
API Response Time (p95)	< 500ms
Insight Generation Time	< 30s for 100K rows
Database Query Time (p95)	< 100ms
Uptime	99.9%
Error Rate	< 0.1%
Test Coverage	> 80%

13.3 Business Metrics

Metric	Target (Year 1)
Total Customers	100
MRR (Monthly Recurring Revenue)	$50K

Metric	Target (Year 1)
Customer Acquisition Cost	< $500
Lifetime Value	> $5000
Churn Rate	< 5% monthly
NPS (Net Promoter Score)	> 50

14. Documentation Deliverables

14.1 Technical Documentation

1. **API Documentation**
   - Swagger/OpenAPI specs
   - Authentication guide
   - Endpoint reference
   - Code examples
2. **Database Documentation**
   - Schema diagrams
   - Table relationships
   - Index strategies
   - Migration guides
3. **Deployment Documentation**
   - Setup instructions
   - Configuration guide
   - Troubleshooting
   - Scaling guide

14.2 User Documentation

1. **Quick Start Guide**
   - 5-minute setup
   - First data upload
   - Understanding insights
2. **User Manual**
   - Feature overview
   - Step-by-step tutorials
   - Best practices
   - FAQ
3. **Video Tutorials**
   - Platform overview (5 min)
   - Uploading data (3 min)
   - Using the chatbot (4 min)

- Creating reports (6 min)

14.3 Developer Documentation

1. **Architecture Overview**
   - System design
   - Component interaction
   - Data flow diagrams
2. **Contributing Guide**
   - Code style
   - Git workflow
   - Testing requirements
   - PR process
3. **API Integration Guide**
   - Authentication
   - Webhooks
   - Custom integrations

---

15. Risk Management

15.1 Technical Risks

Risk	Probability	Impact	Mitigation
Data loss	Low	Critical	Automated backups, replication
Performance degradation	Medium	High	Caching, query optimization, monitoring
Security breach	Low	Critical	Multi-layer security, audits, encryption
LLM hallucinations	Medium	Medium	Confidence scoring, user feedback, validation
Template drift	Medium	Medium	Version control, validation, testing

15.2 Business Risks

Risk	Probability	Impact	Mitigation
Slow adoption	Medium	High	Strong marketing, free tier, case studies
Competitor advantage	Medium	High	Rapid iteration, unique features
LLM cost escalation	Low	Medium	Caching, cost monitoring, alternative models
Regulatory changes	Low	High	GDPR compliance, legal review

Conclusion

This AI Analytics Platform represents a comprehensive solution for businesses seeking intelligent, automated insights from their data. The dual-deployment strategy (SQLite/DuckDB → PostgreSQL) ensures accessibility for small businesses while providing a clear path to enterprise scale.

Key Differentiators:

1. **Template-based intelligence** accelerates time-to-value
2. **AI-powered mapping** reduces manual configuration
3. **Three-tier output** serves different user personas
4. **Seamless scaling** from laptop to cloud

Next Steps:

1. Review and approve this documentation
2. Finalize team structure
3. Begin Sprint 1 (Foundation)
4. Setup development environment
5. Create first industry template

Timeline to MVP: 6 months **Timeline to V1.0:** 9 months

Document Version: 1.0
Last Updated: February 2025
Author: Development Team